PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Методологическое руководство по созданию Canvas-хуков в JavaScript

Введение.
Canvas-хуки позволяют перехватывать и модифицировать вызовы Canvas API (элемент `<canvas>` и контекст рисования) без нарушения оригинальной функциональности. Это используется для задач вроде защиты от отпечатков браузера (fingerprinting) путем внесения шумов в графику, логирования действий, или реализации пользовательских эффектов. Данное руководство описывает архитектуру модульной системы Canvas-хуков на JavaScript: как разделены компоненты, каким образом проксируются вызовы, как регистрируются и выполняются хуки, и как безопасно оборачивать методы Canvas. Мы рассмотрим роли основных модулей (CanvasPatchModule, CanvasPatchHooks, ContextPatchModule), принципы реализации хуков (от оригинальных методов к безопасным обёрткам через `safeInvoke` и `Proxy`), поток выполнения вызовов (от `canvas.getContext` до исполнения хука), детали для ключевых методов Canvas (таких как `drawImage`, `toDataURL`, `putImageData`, `fillText` и др.), а также приведем чеклист для проверки корректности архитектуры.

Архитектура модулей и разделение ответственности

Обзор компонентов. Система Canvas-хуков организована многослойно, с чётким разделением ответственности между несколькими модулями:

- **CanvasPatchModule** – основной инициализатор системы хуков для Canvas. Этот модуль отвечает за запуск всех патчей: он регистрирует все необходимые хуки и применяет патчи к методам Canvas API. Файл `canvas.js` экспортирует функцию-инициализатор `CanvasPatchModule` и объект `CanvasPatchHooks` с реализацией хуков.
- **CanvasPatchHooks** – набор функций-хуков, реализующих конкретные модификации Canvas. Эти функции определяют логику перехвата для отдельных методов (например, `patchToDataURLInjectNoise`, `applyDrawImageHook` и др.). Каждый хук выполняет свою задачу (например, добавление шума, искажение изображения) и может быть зарегистрирован в системе для соответствующего метода Canvas.
- **ContextPatchModule** – модуль, отвечающий за непосредственное перехват и проксирование вызовов контекста рисования Canvas (CanvasRenderingContext2D). Он патчит базовые методы Canvas API (такие как `HTMLCanvasElement.getContext`, `toDataURL`, `toBlob`, а также методы контекста 2D) и внедряет прокси-объекты. ContextPatchModule обеспечивает, чтобы вызовы контекста шли через прокси и подключённые хуки перед тем, как дойти до оригинальной реализации. Проще говоря, этот слой занимается перехватом вызовов и их перенаправлением в систему хуков.
- **Регистраторы хуков** – инфраструктура для хранения и подключения функций-хуков. Глобальный объект (например, `window.CanvasPatchContext` или сокращённо `C`) содержит списки хуков для каждой категории методов и функции-регистраторы для их добавления.

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Например, `C.ctx2DDrawImageHooks` – массив хуков для `CanvasRenderingContext2D.drawImage`, а `C.registerCtx2DDrawImageHook(fn)` – функция, добавляющая новый хук в этот список. Такие регистраторы позволяют модульно подключать/отключать конкретные хуки, не меняя основного кода патчей.

- **Proxy-инфраструктура** – механизм безопасного проксирования 2D-контекста (`CanvasRenderingContext2D`). С помощью Proxy-объекта создается «обёртка» вокруг контекста, перехватывающая вызовы его методов. Цель прокси – обеспечить безопасный вызов (correct binding) методов контекста и предотвратить ошибки типа *Illegal Invocation*, а также в некоторых случаях перенаправить вызов через `safeInvoke` (специальный враппер для безопасного исполнения). Proxy-инфраструктура дополняет систему хуков, гарантируя, что даже при нештатном использовании методов (например, сохранении ссылки на метод и вызове вне контекста) всё продолжит работать корректно.

Ниже мы подробнее разберём роль каждого из этих компонентов.

CanvasPatchModule – инициализация патчей

CanvasPatchModule – это входная точка, с помощью которой включаются все Canvas-хуки. При инициализации этот модуль обычно выполняет следующие шаги:

1. **Создание контекстного патча:** вызывает `ContextPatchModule()` для подготовки окружения. Это инициализирует структуру `CanvasPatchContext` (объект `C`) и патчит ключевые методы Canvas (например, переопределяет `HTMLCanvasElement.prototype.getContext`). Таким образом, уже на этапе получения контекста (`getContext`) система начинает перехватывать вызовы.
2. **Регистрация функций-хуков:** используя экспортируемый объект `CanvasPatchHooks`, CanvasPatchModule регистрирует все необходимые хуки через функции `C.register...Hook`. Например, `C.registerCtx2DDrawImageHook(CanvasPatchHooks.applyDrawImageHook)` подключает хук для `drawImage`, `C.registerHtmlCanvasToDataURLHook(CanvasPatchHooks.patchToDataURLInjectNoise)` – для `toDataURL`, и т.д.. На этом этапе определяется, какие именно функции будут выполнены при перехвате соответствующих вызовов Canvas API.
3. **Применение патчей к методам:** после заполнения списков хуков CanvasPatchModule вызывает функции применения патчей, которые заменяют оригинальные методы на цепочки вызовов хуков. Например, `C.applyCanvasElementPatches()` пропатчит методы `toDataURL`, `toBlob` и `getContext` у `HTMLCanvasElement`, а `C.applyCtx2DContextPatches()` – пропатчит методы у `CanvasRenderingContext2D` (`drawImage`, `putImageData`, `fillText` и т.п.). В результате, вызовы этих методов будут направляться через обёртки, вызывающие зарегистрированные хуки.

Таким образом, CanvasPatchModule связывает всё вместе: запускает патчинг (через ContextPatchModule) и настраивает, какие хуки должны отрабатывать на какие методы. По завершении инициализации, **Canvas API "обёрнут" нашей инфраструктурой**, но для внешнего кода это незаметно – разработчик продолжает вызывать `canvas.getContext`, `context.drawImage` и прочие методы как обычно, просто теперь они проходят через наши перехваты.

CanvasPatchHooks – реализация логики хуков

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

CanvasPatchHooks – это коллекция функций, каждая из которых реализует конкретный хук для Canvas. Эти функции содержат непосредственную логику модификации поведения Canvas API. При проектировании хуков соблюдаются следующие правила:

- **Чистое предназначение:** каждый хук решает одну четко обозначенную задачу (например, добавление шума к изображениям, искажение цветовой гаммы, лёгкое размытие, логирование параметров вызова и т.д.). Логика должна быть *минимальной и изолированной*, не смешивающей разные эффекты. Это упрощает отладку и замену отдельных хуков при необходимости.
- **Без побочных эффектов и идемпотентность:** повторный вызов одного и того же хука на одних и тех же данных не должен накапливать эффект или приводить к непредсказуемым изменениям. Хук должен стремиться изменить только то, что требуется по его задаче, и помечать, что изменение уже внесено (например, флагом на объекте контекста), чтобы не примениться повторно. В частности, если модификация уже сделана (например, данные пикселей уже зашумлены), повторный вызов хука не должен дублировать шум.
- **SafeInvoke для оригинальных методов:** внутри хука при обращении к оригинальным методам Canvas (например, вызов оригинального `getImageData`, `fillText` и др.) необходимо использовать безопасный враппер (`safeInvoke`). Это гарантия, что вызов произойдет с корректным `this` и не вызовет *Illegal invocation* ошибок. Подробно `safeInvoke` мы рассмотрим ниже, но здесь важно, что **никакой хук не вызывает оригинальный метод напрямую** через `origFunc(...args)`, а делает это через обёртку: `safeInvoke(origFunc, this, args)` или похожий механизм.
- **Минимальное влияние на сигнатуру:** хук-обёртка должен, по возможности, возвращать тот же результат, что и оригинальный метод, и принимать те же аргументы (или совместимые). Хук не должен менять тип возвращаемых данных или порядок параметров – он как бы *прозрачен* для остальной системы. Например, если `ctx.drawImage()` по спецификации ничего не возвращает, хук после выполнения тоже не должен возвращать ничего особенного (кроме случая, когда он сознательно подменяет результат функции вроде `measureText`).
- **Документирование и поддержка:** каждый хук сопровождается комментарием или JSDoc с описанием его цели, параметров и возвращаемого значения. Важно фиксировать, в каком порядке и где регистрируется данный хук (например: *«этот хук должен регистрироваться **до** базового шума, но **после** X»*), чтобы при изменении порядка регистрации система функционировала правильно.

Примеры функций из `CanvasPatchHooks`:

- `patchToDataURLInjectNoise`: добавляет шум к содержимому Canvas перед получением DataURL (для маскировки отпечатка изображения).
- `patchCanvasIHDRHook`: изменяет метаданные PNG (IHDR) в DataURL, чтобы, например, нормализовать размеры канваса, влияющие на хеш изображения.
- `applyDrawImageHook`: выполняет комплекс пост-обработок после вызова `drawImage` – может объединять несколько эффектов (добавляет шум, пикселизацию, смещение цветов, размытие, искажение волн) к нарисованному изображению.

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

`addCanvasNoise` (упоминается как `patch2DNoise` или `addCanvasNoise`): применяется при вызо `putImageData` или при чтении данных, чтобы внести шум на уровне пикселей.

- `measureTextNoiseHook`, `fillTextNoiseHook`, `strokeTextNoiseHook`: вносят детерминированные искажения при измерении текста и рисовании текста, основываясь на тексте (например, добавляют *стабильный шум*, зависящий от строкового содержимого, чтобы каждый одинаковый текст искажался предсказуемо одинаково).

CanvasPatchHooks – это **«библиотека эффектов»** для Canvas API. Эти функции сами по себе ничего не патчат; они становятся активными только после регистрации в системе (через `C.register...Hook`). Таким образом, можно конфигурировать поведение: добавляя или убирая определенные хуки из регистрации, мы меняем то, как Canvas будет себя вести.

ContextPatchModule – проксирование контекста и патчинг методов

ContextPatchModule отвечает за низкоуровневое перехватывание вызовов Canvas API. Он выполняет непосредственно «подмену» методов Canvas на обёртки, которые вызовут нужные хуки. Основные задачи ContextPatchModule:

- **Инициализация структуры хуков:** при запуске ContextPatchModule создаётся (или очищается) глобальный объект `CanvasPatchContext` (сокращённо `C`) для хранения списков хуков. Все массивы `C.htmlCanvasGetContextHooks`, `C.ctx2DDrawImageHooks` и т.д. либо создаются, либо сбрасываются в пустые (чтобы повторный запуск не дублировал хуки). Это обеспечивает, что система готова к регистрации новых хуков, а предыдущие регистрации не дублируются – важное условие идемпотентности запуска.
- **Патчинг `canvas.getContext`:** HTMLCanvasElement.prototype.getContext переопределяется, чтобы перехватывать получение контекста рисования. Новый `getContext` вызывает оригинальный метод, получает контекст (например, `CanvasRenderingContext2D`), а затем, если тип контекста `'2d'`, оборачивает его в Proxy через `createSafeCtxProxy`. Кроме того, на этом этапе могут вызываться *hooks for getContext*, если они зарегистрированы: после оборачивания контекста прокси, ContextPatchModule проходит по списку `C.ctx2DGetContextHooks` и вызывает каждый хук, передавая ему полученный контекст и параметры вызова. Эти хуки могут выполнить дополнительную настройку контекста. Например, хук `patch2DNoise` помечает контекст, что к нему применён шум, и может переопределить некоторые методы контекста (как `getImageData`) для внедрения шума. В итоге `canvas.getContext('2d')` возвращает не оригинальный контекст, а его прокси-обёртку, уже готовую к перехвату вызовов.
- **Патчинг методов CanvasRenderingContext2D:** Основные методы 2D-контекста, которые необходимо перехватывать, заменяются на обёртки. ContextPatchModule либо использует вспомогательную функцию `patchCtx2D`, проходящую по списку методов, либо индивидуально заменяет методы. В нашем случае используется подход «цепочек хуков». Например, ContextPatchModule вызывает `C.applyCtx2DContextPatches()`, внутри которой определен массив методов для патчинга: `measureText`, `fillText`, `strokeText`, `putImageData`, `drawImage`. Для каждого из них выполняется замена функции на новую, которая пройдёт по зарегистрированным хукам:
    - Пример: при патчинге `CanvasRenderingContext2D.prototype.drawImage`, новый метод будет сначала вызывать все функции из списка `C.canvas2DNoiseHooks` (например,

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

общий шум для всех рисунков) и `C.ctx2DDrawImageHooks` (специфичные хуки для drawImage). Каждый хук может изменить аргументы (например, подменить изображение или координаты) или выполнить какие-то действия до/после рисования. После прохождения хуков, обёртка вызовет оригинальный метод `drawImage` (с изменёнными или оригинальными аргументами). Наконец, она вернёт результат оригинального метода (для drawImage это обычно `undefined`).
  - Аналогично патчатся `fillText/strokeText` (подключая хуки текста), `measureText` (хуки, влияющие на измерения текста), `putImageData` (хуки шума для пиксельных данных). Все такие патчи помечаются специальным флагом (например, свойство функции `CanvasGlobal = true` или флаги на самом контексте) для предотвращения двойного патчинга.
  - **Асинхронные методы:** `HTMLCanvasElement.toBlob` (и аналогичный OffscreenCanvas.convertToBlob) патчатся отдельной функцией `chainAsync`, так как они возвращают результат асинхронно (через callback или Promise). Новый `toBlob` оборачивает оригинальный вызов: если разработчик использует колбэк-версию (`canvas.toBlob(callback)`), то обёртка просто вызывает оригинал через `safeInvoke`. Если же вызов сделан без колбэка (наша реализация предполагает возвращение Promise), то обёртка создаёт Promise, вызывает оригинальный `toBlob`, передавая `resolve` в качестве колбэка, и возвращает Promise. Для OffscreenCanvas.convertToBlob – сразу возвращается promise и можно навесить `.then`-хуки для обработки Blob (пример хука `patchConvertToBlobInjectNoise` использует `.then` для пост-обработки Blob).
- **Патчинг методов HTMLCanvasElement:** Помимо getContext и toBlob, ContextPatchModule патчит метод `HTMLCanvasElement.toDataURL`. Этот метод синхронный, поэтому используется механизм цепочки (chain). После подключения хуков (список `C.htmlCanvasToDataURLHooks`), новый `toDataURL` при вызове проходит по всем хукам, давая им возможность изменить аргументы (например, формат или параметры качества) и выполнить доп. действия, а затем вызывает оригинальный `toDataURL` с помощью безопасного вызова. Таким образом, можно реализовать, например, добавление шума непосредственно перед получением DataURL и изменение некоторых байтов в результате.

В коде ContextPatchModule предусмотрены **предохранители**:

- Функция `guardInstance(proto, this)` проверяет, что текущий `this` действительно является экземпляром нужного класса (например, что `this` – CanvasRenderingContext2D для методов контекста, или HTMLCanvasElement для методов элемента). Если вдруг метод патча вызван в неправильном контексте, система не пытается применять хуки, а просто делегирует к оригиналу через safeInvoke. Это защищает от ситуаций, когда кто-то мог вручную вызвать наш метод с поддельным контекстом.
- Флаги рекурсии: при патчинге методов вводятся скрытые свойства на контексте, например `__isCtx2D_drawImage` или `__isChain_toDataURL`, которые отмечают, что мы уже внутри перехваченного вызова. Если в ходе выполнения хука произойдет повторный вызов того же метода (например, хук сам вызывает `drawImage`), благодаря флагу обёртка узнает о рекурсии и вызовет оригинал напрямую (через safeInvoke), избегая бесконечного цикла. После выполнения флаг сбрасывается.

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Пометка пропатченных функций: каждому замененному методу присваивается свойство `CanvasGlobal = true`, чтобы при повторной инициализации (или загрузке другого скрипта с такими же патчами) не патчить метод повторно. Функция-регистратор тоже проверяет это свойство, избегая дублей.

Итого, ContextPatchModule создает уровень перехвата, где оригинальные методы Canvas элемент/контекста заменены на функции-обёртки. Эти обёртки при вызове проходят через списки зарегистрированных хуков, выполняют их, затем вызывают оригинальную логику (при необходимости – безопасно) и возвращают результат. Внешний код не замечает подмены, так как интерфейс методов и возвращаемые значения сохраняются.

Proxy для CanvasRenderingContext2D (безопасный контекст Proxy)

Отдельного внимания заслуживает Proxy-инфраструктура – `createSafeCtxProxy`, которая оборачивает объект контекста 2D. Прокси используется, чтобы решить две основные задачи:

1. **Безопасный вызов методов (Illegal Invocation):** Методы встроенных API, таких как CanvasRenderingContext2D, при потере привязки к объекту могут выбрасывать ошибку *Illegal invocation*. Например, если сохранить ссылку на метод `fillText` в переменную и вызвать ее: `const f = ctx.fillText; f("Hello", 0, 10);` – это приведет к ошибке `TypeError: Illegal invocation`, потому что внутренне метод ожидает, что `this` указывает на сам контекст. Прокси-обёртка решает эту проблему: при попытке получить функцию из контекста, она возвращает не оригинальную функцию, а **обёртку**, которая при вызове сама подставит корректный контекст.
2. **Выборочный перехват ключевых методов:** Мы можем настроить Proxy так, чтобы для некоторых методов выполнять дополнительную логику (например, всегда использовать `safeInvoke`), а остальные методы вызывать напрямую. Это оптимизирует работу: не все методы CanvasContext требуют особой обработки, и нет смысла перегружать их лишними проверками.

Реализация `createSafeCtxProxy(ctx)` на практике выглядит так:

```js
function createSafeCtxProxy(ctx) {
  if (!(ctx instanceof CanvasRenderingContext2D)) return ctx;
  const proto = CanvasRenderingContext2D.prototype;

  function safeInvoke(methodName, target, args) {
    try {
      return target[methodName](...args);
    } catch (e) {
      // Если вылетел Illegal invocation, пробуем вызвать через call на прототипе
      if (e instanceof TypeError && /Illegal invocation/.test(e.message)) {
        return proto[methodName].call(target, ...args);
      }
      throw e;
    }
  }

  return new Proxy(ctx, {
    get(target, prop, receiver) {
      const orig = Reflect.get(target, prop, receiver);
      if (typeof orig === 'function') {
        return function(...args) {
          // Для некоторых методов используем safeInvoke
          if (prop === 'fillText' || prop === 'strokeText' ||
```

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

```
            prop === 'measureText' || prop === 'getImageData' ||
            prop === 'putImageData' || prop === 'drawImage') {
          return safeInvoke(prop, target, args);
        }
        // Остальные вызываем напрямую
        return orig.apply(target, args);
      };
    }
    return orig;
  }
});
}
```

Фрагмент кода Proxy-прокладки для контекста Canvas. Здесь:

- Если свойство контекста является функцией, возвращается новая функция-обёртка. Для определенного набора методов (`fillText`, `strokeText`, `measureText`, `getImageData`, `putImageData`, `drawImage`) эта обёртка вызывает оригинальный метод через `safeInvoke`. Для всех остальных методов обёртка просто делает `orig.apply(target, args)`, т.е. вызывает оригинальный метод без дополнительной логики.
- Если запрашивается не-функция (например, свойство `canvas` или `font`), Proxy возвращает значение без изменений – `Reflect.get` берет его у оригинального объекта.

SafeInvoke внутри Proxy: В приведенном коде видно внутреннюю функцию `safeInvoke(methodName, target, args)`. Она делает попытку вызвать метод напрямую, и если ловит ошибку *Illegal invocation*, то выполняет вызов через прототип с явным привязкой `call`. Таким образом, если кто-то вызовет метод вне контекста или произойдет другой сбой, мы подстрахуемся вторым способом вызова. Кроме того, в контексте Proxy, `safeInvoke` гарантирует вызов на оригинальном объекте `target` (а не на Proxy), чтобы обойти возможные ловушки Proxy при внутренних вызовах.

Что оборачиваем и зачем: Оборачиваются *все методы CanvasRenderingContext2D*, но **специально обрабатываются** те методы, где велик риск ошибки привязки или требуется особый подход:

- Методы рисования текста (`fillText`, `strokeText`, `measureText`) – часто используются и могут вызываться с потерей контекста; их обёртка через safeInvoke гарантирует, что шрифт и контекст привязаны правильно. Плюс, эти методы могут быть захвачены хук-системой для добавления шумов (например, изменение возвращаемого `TextMetrics` в случае `measureText`), поэтому лишний слой безопасности не помешает.
- Методы работы с пикселями (`getImageData`, `putImageData`) – также могут быть вызываны в отрыве, и плюс они **могут быть переопределены** нашей же системой (например, `patch2DNoise` заменяет `getImageData` на обёртку с шумом). SafeInvoke позволит вызвать оригинал даже если его заменили.
- `drawImage` – ключевой метод рисования изображений, мы хотим чтобы он тоже был всегда корректно привязан, особенно учитывая что у нас есть собственные хуки на нём.

Избежание Illegal Invocation: Итак, Proxy-прокладка предотвращает *Illegal invocation* не только при прямых вызовах методов, но и в более коварных сценариях, когда методы передаются куда-то. Например, если библиотека клонирует методы CanvasRenderingContext2D или делает `{ draw: ctx.drawImage }` и вызывает `draw()`, то наш Proxy вернёт функцию-обёртку, которая внутри сделает `orig.apply(ctx, args)` – `ctx` сохраняется через замыкание, поэтому

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

вов пройдет успешно. Без Proxy такой код упал бы с TypeError. SafeInvoke с вызовом через `call` на прототипе – это дополнительная страховка на случай, если даже `apply` по каким-то причинам не сработает (хотя в большинстве случаев `orig.apply(target, args)` достаточно).

Proxy и производительность: Мы оборачиваем только **контекст 2D**, а не сами `<canvas>` элементы, и лишь методы, а не свойства. Чтение свойств (ширина, высота, текущий state) идет напрямую к оригиналу. Это значит, что накладные расходы минимальны и не влияют на отрисовку. InstanceOf и прочие проверки также работают, так как прокси не трогает внутренний прототип (например, `proxyCtx instanceof CanvasRenderingContext2D` должно вернуть true, потому что Proxy, не переопределяя ловушку `getPrototypeOf`, сообщает настоящим прототипом оригинального объекта).

Хук-регистраторы и списки вызовов

Глобальный объект C (CanvasPatchContext). Как упоминалось, ContextPatchModule создаёт объект `C = window.CanvasPatchContext` для хранения всего, что связано с хуками. В нём присутствуют массивы-хранилища для каждого вида хуков. Например:

- `C.htmlCanvasToDataURLHooks` – список хуков для `HTMLCanvasElement.prototype.toDataURL`.
- `C.ctx2DDrawImageHooks` – список хуков для `CanvasRenderingContext2D.drawImage`.
- `C.canvas2DNoiseHooks` – список хуков шума для 2D-контекста, применяемых к разным методам (например, общий шум для `drawImage` и `putImageData`).
- Аналогичные списки есть для `fillText`, `strokeText`, `measureText`, `toBlob`, а также для WebGL-методов (на случай, если модуль расширяется на WebGL).

Каждый такой список наполняется через соответствующую функцию-регистратор:

```
js

C.registerCtx2DDrawImageHook = fn => typeof fn === 'function' &&
C.ctx2DDrawImageHooks.push(fn);
```

Подобные функции существуют для всех типов хуков: `registerHtmlCanvasToBlobHook`, `registerCtx2DMeasureTextHook`, `registerWebGLGetContextHook`, и т.д.. Они делают простую вещь: если передан параметр `fn` и это функция, добавляют его в нужный массив. Это предохранитель от случайной передачи не-функции, а также единая точка, где можно было бы, например, пометить, что «хук зарегистрирован».

Порядок регистрации имеет значение. Поскольку хуки хранятся в массивах и выполняются последовательно, **важно документировать порядок, в котором регистрируются хуки**. Например, для `toDataURL` сначала регистрируется хук шума, потом хук IHDR, потом объединяющий master-хук. В результате при вызове `toDataURL` эти хуки отработают в заданной очередности. Если поменять их местами, результат может отличаться (например, сначала применится IHDR-патч, а потом шум – что неправильно, т.к. шум может менять пиксели и требовать обновления CRC). Поэтому при добавлении новых хуков или изменении существующих нужно поддерживать осмысленный порядок. В коде желательно фиксировать это в комментариях: "*Hook X must run before Hook Y*".

Лёгкое отключение хуков. Благодаря централизации регистраций, при необходимости можно отключить определенный хук или все сразу. Несколько стратегий:

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Глобальный флаг: Можно внедрить проверку `if (!window.CanvasHookEnabled) return orig.apply(this, args)` в начале обёртки – тогда установка `window.CanvasHookEnabled = false` отключит влияние всех хуков. В рекомендациях упоминалось использование проверки некоего CanvasGlobal для легкой замены/удаления. Это может быть реализовано как глобальный объект с настройками.

- *Удаление из списка:* Убрать функцию из массива `C.ctx2DDrawImageHooks` или очистить массив – тогда она не будет вызываться. Так как сами методы уже пропатчены, важно убирать аккуратно (в идеале до инициализации или переинициализируя модуль).
- *Перепривязка оригиналов:* Если нужно полностью снять патчи, можно хранить оригиналы методов (в коде оригиналы сохраняются, например `const origGetContext = HTMLCanvasElement.prototype.getContext`, `origDrawImage = proto.drawImage` при патчинге) и затем восстанавливать их. Пометка `CanvasGlobal` на методах помогает определить, что метод был изменен, и потенциально использовать это для восстановления.

Итак, **регистраторы хуков обеспечивают модульность**: логика хуков отделена от места их применения. Можно иметь несколько файлов с реализацией хуков (например, отдельный файл для WebGL-хуков), которые через API регистратора подключаются к общей системе, не вмешиваясь в сам механизм патчей. Это согласуется с принципом "*каждый файл хуков отвечает за свою область (Canvas, WebGL и т.п.)*".

Принципы разработки и реализации Canvas-хуков

При создании новых хуков для Canvas API следует придерживаться методического подхода. Ниже приведён **пошаговый процесс разработки хука** и ключевые принципы, гарантирующие корректность и безопасность патчей.

Шаги создания хука (workflow)

1. **Выбрать целевой метод:** Определите, какой метод Canvas API требует перехвата. Обычно выбор основан на анализе того, какие действия могут использоваться для нежелательного поведения (например, снятие отпечатков). Примеры: `HTMLCanvasElement.toDataURL`, `CanvasRenderingContext2D.drawImage`, `getImageData`, `fillText` и т.д..
2. **Определить цель хука:** Что вы хотите сделать при перехвате этого метода? Варианты: добавить шум (noise) к выходным данным, исказить изображение (distortion, blur, pixelate, color shift), измерить параметры вызова (логирование), предотвратить вызов или подменить значение (например, для fingerprinting некоторые параметры WebGL могут быть подменены). Чётко сформулируйте задачу: *«При каждом вызове `ctx.drawImage` добавлять случайный шум к нарисованному изображению»* или *«При вызове `canvas.toDataURL` внедрять искажение, не меняя размеров изображения»*.
3. **Сохранить оригинал метода:** Перед тем как патчить метод, обязательно сохранить ссылку на исходную функцию. Это обычно делается один раз в ContextPatchModule (например, `const origDrawImage = CanvasRenderingContext2D.prototype.drawImage`). Оригинал понадобится, чтобы вызывать его внутри вашей обёртки. Если вы пишете хук не как замену метода, а как функцию в списке хуков, оригинал тоже может быть передан ей как параметр (например, в нашем коде `applyDrawImageHook` принимает `origDrawImage` в аргументах).

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Реализовать чистую обёртку (hook function): Напишите функцию, которая обрабатыв вызов метода. В зависимости от модели (chain или patchMethod) ваша функция может вызываться *вместо* оригинала или *до/после* оригинала. Общие подходы:

- **Pre-processing (до вызова оригинала):** Например, для `toDataURL` вы можете изменить содержимое canvas перед тем, как будет получен DataURL. В этом случае хук выполнит свои действия, а потом вернёт `args` или попросит вызвать оригинал. В коде chain-хука, если хук возвращает массив аргументов, они заменят оригинальные при вызове оригинала.
- **Post-processing (после вызова оригинала):** Например, для `drawImage` логично сначала позволить оригиналу нарисовать изображение, а затем скорректировать нарисованное. В этом случае хук должен вызываться *после* оригинальной функции и, возможно, заменять возвращаемое значение или сам факт рисования. В нашей системе post-обработка достигается двумя способами: либо хук встроен в обёртку *внутри patchMethod* (как `applyDrawImageHook` – он сам вызывает `origDrawImage`, затем выполняет эффекты), либо в chain-модели хук вызывается после оригинала, если использовать `patchMethod` с проверкой `if (res !== undefined) return res` (см. WebGL часть, но схожий принцип).
- **Подмена результата:** Если ваш хук преследует цель изменить возвращаемое значение функции (например, `measureText` возвращает объект TextMetrics – можно слегка изменить ширину), то хук может вернуть новое значение. В цепочке хуков, если функция-хук вернула не `undefined`, то эта возвращаемая величина интерпретируется как «результат функции», и оригинал может не вызываться (в реализации `patchMethod`: *если hook вернул не undefined, вернуть его, не вызывая orig*).
- **Сохранение сигнатуры:** Ваша функция должна принимать те же аргументы, что и оригинал (в chain-хуках — иногда первым параметром идёт оригинальный результат или функция). Например, `function measureTextNoiseHook(width, text)` – не совсем верно, лучше `function measureTextNoiseHook(origMetrics, text)` если предполагать, что она вызовется после получения результата. Уточните контекст вызова вашего хука.

После выполнения нужной логики (например, добавили шум в пиксельные данные или вычислили исправленное значение), **верните результат**: либо изменённые аргументы (для передачи следующему шагу), либо конкретное значение/объект, если хотите прервать дальнейшую цепочку, либо ничего (undefined), если ваша обработка завершена и вы полагаетесь на оригинал.

5. **Зарегистрировать хук:** В соответствующем месте инициализации (CanvasPatchModule) добавьте ваш хук в систему: вызовите `C.register...Hook(yourHookFunction)` для нужного списка. Например, если вы сделали хук на чтение данных пикселей, зарегистрируйте его через `C.registerCtx2DGetImageDataHook(fn)` или, в нашей структуре, если getImageData обрабатывается через общий шумовой хук, возможно `C.registerCtx2DAddNoiseHook(fn)`.
6. **Применить/обновить патч метода:** Убедитесь, что целевой метод действительно перехватывается. Если раньше он не был включен в патчинг, добавьте его. В `applyCtx2DContextPatches` можно включить новый метод и связать с вашим списком хуков. Если используется общая цепочка (chain) для группы методов, убедитесь, что при инициализации она вызывается.

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Примечание: В нашем случае большинство важных методов уже патчены. Например, чтобы начать перехватывать `CanvasRenderingContext2D.getImageData`, есть два подхода:

- Добавить `getImageData` в список методов patchCtx2D (как это сделано для `putImageData`), и завести `C.ctx2DGetImageDataHooks`.
- Или применять шум через хук на этапе getContext (как сделано `patch2DNoise`), который переопределяет `res.getImageData` у конкретного контекста. В документации указано, что getImageData требует патча, значит нужно проверить, что он реализован. **Проверка реализации** – часть вашей работы при добавлении нового хука.

7. **Тестирование:** После подключения хука, протестируйте его в различных сценариях:
   - Вызов целевого метода в штатном режиме (ожидается корректный результат, возможно с вашим эффектом). Например, убедитесь, что `canvas.toDataURL()` возвращает строку, отображающую изображение, и что изображение действительно изменено (например, пиксели отличаются).
   - Граничные случаи: вызов метода вне контекста (например, сохранённой функцией), в разных браузерах, с разными аргументами. В идеале написать unit-тесты, но здесь отмечено, что мы не фокусируемся на них – однако простые проверки вручную очень желательны.
   - Проверьте, **не сломано ли оригинальное поведение**: если отключить эффекты, результат должен быть идентичен исходному. Например, ваш хук `measureTextNoiseHook` не должен вдруг менять тип возвращаемого значения или кидать исключения.

Следуя этим шагам, вы создадите хук, который встроится в общую инфраструктуру без регрессий.

Ключевые принципы проектирования хуков (резюме)

Помимо описанного процесса, используйте такие принципы при проектировании архитектуры хуков:

- **Proxy vs Hooks – разделение задач:** Прокси-объекты предназначены только для безопасного вызова методов и минимальной маршрутизации, **не стоит добавлять в Proxy бизнес-логику хуков**. Все специфические преобразования должны выполняться в функциях-хуках (CanvasPatchHooks). Proxy – это про низкоуровневую правильность вызовов, а хуки – про *что изменить в функционале*. Такая декомпозиция улучшает поддерживаемость: можно менять реализацию хука, не трогая Proxy, и наоборот.
- **Слои ответственности:** Можно мысленно выделить три слоя:
  1. *Контекстный слой перехвата* – ContextPatchModule, выполняющий проксирование и замену методов (т.е. обеспечивающий сам факт, что хуки будут вызваны).
  2. *Слой логики хуков* – CanvasPatchHooks, где реализованы конкретные эффекты.
  3. *Утилиты и общие функции* – вспомогательные функции вроде генерации шума, размытия, смещения цветов, которые могут использоваться несколькими разными хуками. Эти утилиты должны быть изолированы и не знать о Canvas напрямую, чтобы их можно было легко комбинировать.

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Локальность и минимальный охват: Если хук вносит изменения в пиксели, старайтесь затрагивать минимально необходимую область. Например, при перехвате `ctx.drawImage(image, dx, dy, ...)` – добавляйте шум только в тот прямоугольный регион, куда было нарисовано изображение, а не во весь холст. В реализации `applyDrawImageHook` именно так: он вычисляет область (`dx, dy, dw, dh`) и берет `getImageData` только по этой области, применяет эффекты и кладет её обратно.

- **Стабильность и повторяемость:** Если вы вносите случайные искажения (noise), делайте их псевдослучайными, но детерминированными, когда надо. Например, привязывайте генератор случайных чисел к глобальному seed (`window.__GLOBAL_SEED`). В нашем коде видно использование `window._rand` – вероятно, это функция-генератор, который выдаёт последовательность на основе фиксированного зерна. Для текстовых хуков предлагается *stableNoiseFromString* – шум, который зависит от содержимого строки, чтобы один и тот же текст искажался одинаково на разных машинах. Это предотвращает утечку информации о системе (например, разные GPU могли бы давать разные шумы, а так они приведены к общему знаменателю).
- **Безопасность от рекурсии и повторной инициализации:** Используйте флаги (как `__isChain_method`, `CanvasGlobal` property) для защиты от повторного входа и повторного патчинга. Это мы уже обсудили – принцип в том, чтобы система хуков работала *идемпотентно*: один и тот же патч можно применить дважды без удвоения эффекта, а вызов хука, вызывающий сам себя, не приведет к бесконечному циклу.
- **Декомпозиция сложных эффектов:** Если требуемый эффект сложен, **разбейте его на несколько хуков или утилит**. Например, в нашем pipeline для `drawImage` вместо единого громоздкого хука, который делает и шум, и пикселизацию, и размытие, реализовано несколько маленьких функций: `applyStableNoise`, `pixelate`, `colorShift`, `blur`, `waveDistort`, которые применяются последовательно. Они могут вызываться либо в одном хук-функции последовательно (как в `applyDrawImageHook`), либо даже быть разнесены по разным хукам, регистрируемым для `drawImage` (например, можно было бы зарегистрировать отдельные хуки для шума, для пикселизации и т.д., тогда цепочка их вызовов дала бы суммарный эффект). Правильная декомпозиция упрощает замену отдельных шагов (например, заменить алгоритм blur на другой, не трогая шум и цветовой сдвиг).
- **Проверка типа контекста:** Убеждайтесь, что ваш хук выполняется только в уместном контексте. В коде часто встречается проверка `if (!(this instanceof HTMLCanvasElement || ... OffscreenCanvas)) return res;` внутри хуков для canvas, или проверка `type === '2d'` при getContext. Это нужно, чтобы, например, не пытаться применить 2D-шум к WebGL-контексту. Наш guardInstance также работает на это.

Придерживаясь этих принципов, вы получите систему, которая **расширяема, безопасна и поддерживаемая**. Добавление новых хуков не сломает старые, а снятие всех патчей вернет Canvas к нормальному поведению без побочных эффектов.

Поток вызовов в системе Canvas-хуков

Теперь рассмотрим, как происходит **прохождение вызова** через всю нашу инфраструктуру – от момента, когда сторонний код вызывает метод Canvas API, до выполнения наших хуков и

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

получения результата. Здесь мы проследим два ключевых сценария: получение 2D-контекста и вызов методов рисования/чтения через прокси.

1. Перехват `canvas.getContext('2d')`

<small>*Диаграмма: последовательность при вызове getContext*</small>

1. **Пользовательский код:**

```
js

const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');
```

   Разработчик вызывает стандартный метод получения контекста.

2. **Перехват CanvasPatch:** Наш патчированный `HTMLCanvasElement.prototype.getContext` срабатывает вместо оригинального. Эта обёртка (установленная ContextPatchModule) сначала вызывает оригинальный метод, чтобы получить реальный контекст:

```
js

const realCtx = origGetContext.call(canvas, '2d', ...args);
```

   (Используем `call` на случай, если оригинал ожидает правильный `this` – хотя тут `origGetContext` взят с прототипа и `canvas` передан, так что всё ок).

3. **Создание Proxy контекста:** Поскольку тип запрошенного контекста – `'2d'`, обёртка вызывает `createSafeCtxProxy(realCtx)`. Это вернет **проксированный контекст** (`proxyCtx`), который выглядит как обычный CanvasRenderingContext2D, но с ловушками на методы. Внутри `createSafeCtxProxy` происходят вещи, описанные ранее: устанавливаются обёртки на функции `fillText`, `drawImage` и др., через safeInvoke. Если почему-то `realCtx` не является инстансом CanvasRenderingContext2D (например, `getContext('webgl')`), прокси не применяется и возвращается оригинал.

4. **Вызов хуков getContext:** После получения `proxyCtx`, обёртка getContext запускает хуки, зарегистрированные для получения контекста:

   - Сначала `C.htmlCanvasGetContextHooks` – хуки верхнего уровня на сам вызов getContext. (В нашей конфигурации обычно не используются или используются для логирования.)
   - Затем, если контекст 2D, перебирает `C.ctx2DGetContextHooks`. Именно сюда был зарегистрирован, например, хук `patch2DNoise`. Этот хук выполняется:

```
js

res = hook.call(canvas, res, '2d', ...args) || res;
```

     Он получает на вход (через `res` или `this`) объект контекста. `patch2DNoise`, судя по коду, ставит флаг `res.__Noise__ = true` и переопределяет `res.getImageData` на свою функцию, которая при каждом вызове добавляет шум к возвращаемым данным (например, меняет несколько битов) – тем самым готовит контекст к "шумному" поведению. Если

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

хук возвращает измененный объект контекста, обёртка зафиксирует это, хотя обыч… такие хуки возвращают либо тот же контекст, либо новый Proxy. (В нашем случае, `patch2DNoise` после установки флагов возвращает тот же `res`.).
- Если бы был webGL контекст, вызвались бы `C.webglGetContextHooks` аналогично, но мы фокусируемся на 2D.

5. **Возврат значения:** Обновлённый (и, возможно, модифицированный хуками) контекст прокси возвращается пользователю:

```js
const ctx = canvas.getContext('2d'); // ctx теперь Proxy
```

Для внешнего кода `ctx` выглядит и ведёт себя как CanvasRenderingContext2D, но в действительности все его методы завёрнуты. Например, `ctx.fillText === CanvasRenderingContext2D.prototype.fillText` будет ложью, так как это не оригинальный метод, а Proxy-обёртка. Однако это прозрачно: devtools могут даже не показать разницы, а instanceOf работает.

6. *(Опционально)* **OffscreenCanvas:** Если вызывается `offscreenCanvas.getContext('2d')`, действует тот же механизм: в ContextPatchModule патчится OffscreenCanvas.getContext аналогично HTMLCanvasElement.getContext. Он тоже возвращает Proxy и вызывает (возможно) `C.offscreenGetContextHooks` и затем `C.ctx2DGetContextHooks` – то есть, OffscreenCanvas 2D-рендеринг тоже попадает под наши хуки.

Итог: После `getContext` у нас есть **проксированный контекст**, отмеченный, что к нему применены шумовые патчи (через флаг), и готовый перехватывать дальнейшие вызовы рисования/получения данных.

2. Вызов методов контекста через Proxy и хуки

Рассмотрим теперь, что происходит, когда пользовательский код начинает работать с полученным контекстом. В качестве примера возьмём популярные действия:

- Рисование изображения: `ctx.drawImage(img, x, y)`
- Получение данных: `ctx.getImageData(x, y, w, h)`
- Прочие, вроде текста: `ctx.fillText("Hello", x, y)`

Сценарий: `ctx.drawImage(image, dx, dy)` (с добавлением шума и эффектов):

1. **Пользовательский код:**

```js
ctx.drawImage(imageElement, 10, 20);
```

Вызывает метод рисования на прокси-контексте.

2. **Proxy перехват (`get` trap):** Поскольку `ctx` – Proxy, при доступе к свойству `drawImage` срабатывает ловушка `get`. Proxy возвращает функцию-обёртку:

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

```js
const drawImageFn = proxyCtx.drawImage; // здесь get trap возвращает function(...args) { ... }
drawImageFn.call(proxyCtx, imageElement, 10, 20);
```

(На практике при прямом вызове `ctx.drawImage()` JavaScript сам берёт функцию и вызывает, что эквивалентно вышеуказанному). Внутри этой обёртки Proxy проверит имя метода `prop === 'drawImage'` и обнаружит, что для него нужно использовать `safeInvoke`. Таким образом, Proxy *не вызывает оригинал сам*, а делегирует в `safeInvoke('drawImage', target, args)`.

3. **Вызов через safeInvoke:** Функция `safeInvoke(prop, target, args)` внутри Proxy попытается выполнить оригинальный метод на оригинальном объекте контекста:

```js
try {
  return target['drawImage'](...args);
} catch(e) {
  if (e instanceof TypeError && /Illegal invocation/.test(e.message)) {
     return proto['drawImage'].call(target, ...args);
  }
  throw e;
}
```

Здесь `target` – это реальный CanvasRenderingContext2D (не Proxy). Когда Proxy создавался, он сохранил `target = realCtx`. Таким образом, `target['drawImage']` – это **пропатченная нами функция drawImage** (ведь мы её заменили на этапе ContextPatchModule). Напомним: в `applyCtx2DContextPatches` мы переопределили `CanvasRenderingContext2D.prototype.drawImage` на свою обёртку, которая запускает цепочку хуков. Значит сейчас `target['drawImage'](...args)` вызывает *нашу цепочку* (chain) для drawImage. Отлично, так и задумывалось.

Если бы этот вызов бросил `Illegal invocation` (что маловероятно, так как мы правильно передали `target`), был бы fallback через `proto.drawImage.call(target, ...)` – тоже на нашу обёртку, только обходя, если что, Proxy, но в целом это запасной путь.

4. **Выполнение обёртки drawImage (chain hooks):** Теперь мы внутри замененной реализации `CanvasRenderingContext2D.drawImage` из ContextPatchModule. Этот код примерно такой:

```js
proto.drawImage = function(...args) {
  const flag = '__isCtx2D_drawImage';
  if (this[flag]) return safeInvoke(origDrawImage, this, args);
  this[flag] = true;
  try {
    for (const hook of [...C.canvas2DNoiseHooks, ...C.ctx2DDrawImageHooks]) {
      try {
        const nextArgs = hook.apply(this, args);
        if (nextArgs && Array.isArray(nextArgs)) {
          args = nextArgs; // хук вернул новые аргументы
        }
      } catch(e) {
        console.error('[CHAIN HOOK ERROR drawImage]', e);
      }
    }
    return origDrawImage.apply(this, args);
  } finally {
    this[flag] = false;
```

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

```
  }
};
```

(Синтезировано на основе кода patchCtx2D и patchMethod).
Пояснения:

- Если `__isCtx2D_drawImage` уже true, значит мы рекурсивно попали, тогда сразу `safeInvoke` оригинальный drawImage (который у нас сохранен как `origDrawImage`) без хуков.
- Устанавливаем флаг и проходим по списку хуков. У нас два списка объединены: общий шум (`canvas2DNoiseHooks`) и специальные хуки drawImage (`ctx2DDrawImageHooks`). Предположительно, в `canvas2DNoiseHooks` может быть хук `addCanvasNoise` (который просто модифицирует что-то независимо от конкретного метода), а в `ctx2DDrawImageHooks` – хук `applyDrawImageHook`.
- Каждый хук вызывается с текущим контекстом как `this` и исходными аргументами `args`. Если хук возвращает массив (изменённые аргументы), мы используем их для следующих хуков и оригинала. Если хук выбросил ошибку – ловим, логируем, но не прерываем цепочку.
- После хуков вызывается сохраненный `origDrawImage` (оригинальная функция рисования, которая непосредственно рисует изображение на канвас) с (возможно) изменёнными аргументами. Результат (для drawImage обычно undefined) сохраняется для возврата.
- Флаг очищается в `finally`.

Теперь посмотрим, что делают сами хуки:

- **Хук шума** (`addCanvasNoise`): может ничего не возвращать (undefined), тогда `args` не меняются, но хук внутри, возможно, прямо работает с контекстом. Например, *другой вариант реализации шума:* можно было бы передать `ctx` и координаты, и внутри хука самостоятельно вызвать `ctx.getImageData`/`ctx.putImageData` вокруг рисуемой области. Однако наш дизайн предполагает, что noise-хук или выполнит что-то отдельно, или вернёт модифицированные args (маловероятно в данном случае).
- **Хук `applyDrawImageHook`:** согласно коду, он реализован как пост-обработка:

  ```js
  function applyDrawImageHook(origDrawImage, ...args) {
    const result = origDrawImage.apply(this, args);
    // ... эффекты ...
    return result;
  }
  ```

  Но подождите, как же он вписывается в цепочку? Дело в том, что `applyDrawImageHook` в нашем коде регистрируется особым образом: в registration мы видим `C.registerCtx2DDrawImageHook(CanvasPatchHooks.applyDrawImageHook)`, а при патчинге drawImage мы комбинируем `canvas2DNoiseHooks` и `ctx2DDrawImageHooks`. Вероятно, `canvas2DNoiseHooks` содержит общий noise, а `ctx2DDrawImageHooks` – наш applyDrawImageHook, который, судя по сигнатуре, принимает `origDrawImage` первым аргументом. Но при вызове `hook.apply(this, args)` он не получит `origDrawImage` автоматически. Здесь накладывается, что `patchMethod` (другая реализация) передавала

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

`orig` в hook, а `patchCtx2D` нет. Вероятно, `applyDrawImageHook` реализован не через chain, а через patchMethod (в коде `applyCtx2DContextPatches` просто передает список, и судя по логике, `patchCtx2D` не знает об `orig` параметре). Возможно, мы допустим, что `applyDrawImageHook` не вызывается через `.apply` вообще, а зарегистрирован для использования в другой схеме… либо он был переписан на возвращение нового args.

Тем не менее, у нас есть код: внутри `applyDrawImageHook` (из `canvas.js`) он сразу вызывает `origDrawImage` и потом эффекты. Как он получил `origDrawImage`? Вероятно, в нашем infrastructure `patchMethod` (который используется для WebGL) можно было бы патчить drawImage. Или же `applyDrawImageHook` у нас не используется через chain.

Допустим для упрощения, что `applyDrawImageHook` – это не chain-хук, а *встроенный* hook, но фактически, судя по registration, он все же в chain. Ладно. В любом случае, он делает:

- `origDrawImage.apply(this, args)`: рисует изображение оригиналом.
- Потом: `ctx = this`, определяет `canvasEl = ctx.canvas || ctx` (для OffscreenCanvas видимо).
- Вычисляет границы (если `sw, sh` не заданы, берет полные размеры canvas).
- `const imageData = ctx.getImageData(dx, dy, dw, dh)`.
- Применяет серию фильтров: `applyStableNoise`, `pixelate`, `colorShift`, `blur`, `waveDistort` последовательно к `imageData`. Каждая функция принимает `imageData` (вероятно, возвращая его же с модификацией) – это изолированные утилиты.
- Потом `ctx.putImageData(data, dx, dy)`, возвращая измененные пиксели на холст.
- Возвращает результат оригинального drawImage (который неважен, но пусть `undefined`).
- **Таким образом, изображение рисуется нормальным API, потом сразу перерисовывается/модифицируется пиксельный блок с примененными эффектами.** Визуально это даёт впечатление, будто при рисовании сразу появился шум/искажение, хотя это сделано двумя шагами.

- **Хук шума** (возможно `addCanvasNoise`): Если также подключен, он мог, например, просто добавить небольшой общий шум по всему canvas или по области. Если он выполнится *до* `applyDrawImageHook`, то вероятно, `applyDrawImageHook` затем ещё раз возьмет `getImageData` уже с учётом этого шума (двойной шум). Может, поэтому `applyDrawImageHook` у них один объединяющий. Не будем глубоко, но, скорее всего, `canvas2DNoiseHooks` включал в нашем примере `addCanvasNoise` – функцию, которая добавляет шум ко всему холсту *до* рисования. Может, она не нужна здесь. Допустим, для drawImage они решили комбинировать, но порядок важен: логично сначала нарисовать (оригинал), потом эффекты, поэтому `applyDrawImageHook` делает и рисование, и эффекты. Тогда возникает вопрос – зачем ему быть хуком, а не прямо заменить drawImage? Видимо, для модульности.

После выполнения всех хуков, drawImage-обёртка возвращает `result` оригинала (undefined). Этот result возвращается из `safeInvoke` Proxy и далее наружу.

5. **Возврат в пользовательский код:** Для вызывающего кода `ctx.drawImage(...)` уже завершился. Он ничего не возвращает, как и должно. Но «за кулисами» на canvas изображение нарисовано с шумом и эффектами, благодаря хукам. Важный нюанс: все эти

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

модификации произошли синхронно в том же событии, так что разработчик не видит разницы (кроме измененной картинки).

Другие методы:

- `ctx.getImageData(x,y,w,h)`: Здесь Proxy также завернет вызов через safeInvoke (потому что getImageData входит в список особых методов). SafeInvoke вызывает наш пропатченный `getImageData`. А `getImageData` у нас, вероятно, был переопределен при getContext (через `patch2DNoise`), чтобы всегда добавить шум к возвращаемым данным. В коде `patch2DNoise` видно:

  ```js
  const origGD = res.getImageData.bind(res);
  res.getImageData = function(x,y,w,h) {
    const imgData = origGD(x,y,w,h);
    // пройтись по imgData.data и внести шум посредством rand()
    return imgData;
  }
  ```

  И также помечается `res.__Noise__ = true` чтобы второй раз не делать это (например, если getContext вызвали снова на том же canvas, не переопределять повторно). Таким образом, когда пользователь вызывает `ctx.getImageData(...)`, он получает уже *искаженные пиксели*. Если этот метод используется для снятия отпечатка (что часто делается: рисуют текст или градиент, потом читают пиксели и вычисляют хеш), то благодаря нашему хуку хеш будет изменен.

- `ctx.putImageData(imgData, dx, dy)`: Патчинг putImageData настроен так, что он запускает хуки шума перед размещением данных. В нашем случае `applyCtx2DContextPatches` подключал для `putImageData` список `canvas2DNoiseHooks`. Предположительно, там зарегистрирован `addCanvasNoise` – функция, которая генерирует шум и смешивает с переданным `imgData`. Возможно, реализовано через *post*-логику: вызывается оригинальный `putImageData`, а после него хук проходит по canvas (или по данным) и добавляет шум. Но учитывая, что `putImageData` просто кладет данные, может лучше *до* положить добавить шум к `imgData.data`. Есть два подхода:

  - **Pre:** передать хукам сам массив данных, чтобы они его модифицировали. Если хук возвращает массив (например, `[modifiedImageData, dx, dy]`), то patchCtx2D возьмет их и вызовет orig с ними.

  - **Post:** вызвать orig, а потом наложить шум. Пост-обработка потребует читать обратно данные (дорого) или вспомнить, что положили. Скорее pre-обработка эффективнее.
    Вероятно, addCanvasNoise работает именно с `args` и возвращает новый `args` с шумным ImageData. В pseudo:

    ```js
    function addCanvasNoiseHook(imageData, dx, dy) {
      // перебрать imageData.data, каждому каналу добавить +-1
      return [imageData, dx, dy];
    }
    ```

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Тогда chain увидит Array и обновит args.
Отсутствие явного кода не мешает понять: цель – **убедиться, что любые данные, помещаемые на canvas через putImageData, получают долю шума** (нарушая тем самым возможность создания "чистого" канваса без шумов, если злоумышленник попытается обойти drawImage).

- `ctx.fillText(text, x, y)` / `strokeText`: Proxy будет вызывать их через safeInvoke (в списке), а оригиналы этих методов мы тоже заменили. В `applyCtx2DContextPatches` они есть, с хуками `ctx2DFillTextHooks` и `ctx2DStrokeTextHooks`. Значит обёртка:

```
js

proto.fillText = function(...args) {
  // recursion flag logic...
  for(const hook of C.ctx2DFillTextHooks) {
     try { hook.apply(this, args); } catch(e) { ... }
  }
  return origFillText.apply(this, args);
};
```

(примерно так, если хук не изменяет аргументы).
Цель хуков на текст – **внести стабильное искажение текста**. Например:

- `fillTextNoiseHook`: может немного исказить нарисованный текст. Но как? Можно, например, нарисовать текст, затем получить его ImageData как мы делали с drawImage, добавить шум и вернуть на место. Но это накладно для каждого текста. Может, проще: слегка изменить сам `text` или `font` перед рисованием. Однако менять шрифт нельзя без заметных эффектов. Скорее, эффект: после origFillText вызвать какие-то фильтры на месте текста.
- С другой стороны, *если текст используется для fingerprinting, чаще читают пиксели или measureText, чем смотрят визуально.* Поэтому особенно важно:
  - `measureTextNoiseHook`: этот хук может корректировать ширину. Например, взять результат `TextMetrics` и увеличить/уменьшить width на ± случайные несколько единиц (но детерминированно для данной строки).
    Возможно реализация:

```
js

proto.measureText = function(text) {
  // recursion guard...
  let metrics = origMeasureText.call(this, text);
  for(const hook of C.ctx2DMeasureTextHooks) {
    const res = hook.call(this, metrics, text);
    if(res !== undefined) metrics = res;
  }
  return metrics;
};
```

А `measureTextNoiseHook` может быть:

```
js

function measureTextNoiseHook(metrics, text) {
  const stableOffset = stableRandomFromString(text); // e.g. ±0.1
  metrics.width += stableOffset;
```

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

```
    return metrics;
  }
```

Так для каждого текста измеренная ширина чуть плавает, затрудняя точное определение рендера шрифтов на данной системе, но при повторном измерении того же текста (или на другой странице той же строки) результат будет тот же (стабильно по строке).

- **`fillTextNoiseHook`/`strokeTextNoiseHook`**: могут использовать подход, аналогичный drawImage: после рисования текста получить область текста (скажем, зная приблизительные размеры или используя только что полученный metrics), и немного изменить пиксели (размыть края, поменять несколько пикселей). Либо вариант: *вводить шум через глобальный композитинг:* например, перед вызовом **`fillText`**, настроить **`ctx.globalAlpha`** очень малый и нарисовать на 1px смещенный тот же текст, затем восстановить alpha. Это даст эффект нечеткости/шума, почти незаметно. Но таких трюков сложнее придумать.

Как бы ни были реализованы текстовые хуки, принципиально: **они вызываются в том же chain, не ломают сам процесс рисования текста** (оригинал всегда вызывается), а лишь добавляют небольшой визуальный или метрический шум.

Asynchronous path (`toBlob`): Рассмотрим еще отдельно **`canvas.toDataURL()`** и **`canvas.toBlob()`**:

- **`canvas.toDataURL()`**: Патч произошел через **`chain(HTMLCanvasElement.prototype, 'toDataURL', hooks)`**. В списке **`C.htmlCanvasToDataURLHooks`** у нас несколько хуков: **`patchToDataURLInjectNoise`**, **`patchCanvasIHDRHook`**, **`masterToDataURLHook`**. Алгоритм **`chain`** для sync (см. ранее) запускает все хуки, а затем вызывает оригинал. Однако, как мы заметили, логика этих хуков сложная: первый хук шум добавляет, второй IHDR, третий объединяет результаты. Их реализации:
  - **`patchToDataURLInjectNoise(res, type, quality)`**: в коде видно, что он проверяет, что тип изображения (если не задан или является image/*), получает **`ctx2d = this.getContext('2d')`**, берет оттуда все пиксели, вносит в них шум (путем случайного ±1 к каналам), затем через оригинальный **`ctx2d.putImageData`** кладет их обратно. То есть, **он модифицирует сам canvas**, но никак не использует параметр **`res`**. Затем возвращает **`res`** без изменений. Поскольку chain вызывает оригинал *после* хуков, здесь **`res`** скорее всего undefined (или если chainAccumulator, то оно игнорируется). Идет чистая pre-обработка: холст зашумлен, дальше будет оригинальный toDataURL.
  - Оригинальный **`toDataURL`** выполняется (через safeInvoke) и получает изображение **уже с шумом**.
  - Затем, *теоретически*, должна быть пост-обработка IHDR. Но chain реализован так, что после original уже не вызвать хуки (они были до). Поэтому, вероятно, **`patchCanvasIHDRHook`** и **`masterToDataURLHook`** ожидали другой вызов. Может, их запускали через **`patchMethod`** (который позволяет хукам перехватывать результат). Или они рассчитывали, что **`patchCanvasIHDRHook`** ничего не делает, а **`masterToDataURLHook`** сам вызывает сначала **`patchToDataURLInjectNoise`**, потом original, потом **`patchCanvasIHDRHook`**.
    Судя по их кодам:

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

- `patchCanvasIHDRHook(res, type, quality, newW, newH)`: если PNG и заданы newW/newH, он внутри вызывает `patchCanvasIHDR(canvas, newW, newH)` и возвращает DataURL с исправленным IHDR. Если не применимо – возвращает `res` как было.
- `masterToDataURLHook(res, type, quality)`: вызывает `patchToDataURLInjectNoise.call(this, res, type, quality)`, потом вычисляет `newW`/`newH` (из глобальных или this.width/height) и вызывает `patchCanvasIHDRHook.call(this, res, type, quality, newW, newH)`. MasterHook тем самым объединяет шум и IHDR, ожидая что `res` – это DataURL после оригинала. Значит, `masterToDataURLHook` предполагался вызываться **после** orig и других хуков. Это не соответствует default chain, поэтому, скорее всего, разработчики добавили логику: *в chain, если hook возвращает что-то не undefined, вернуть сразу*. То есть, возможно `masterToDataURLHook` мог перехватить `res` (DataURL) на выходе. Но chain у нас не делает такого для undefined hooks.

Чтобы не путаться: **в итоге** наш patched toDataURL, вероятно, работает так:

- Если masterHook зарегистрирован последним, chain вызовет сначала patchToDataURLInjectNoise (prepares canvas), потом patchCanvasIHDRHook (который, не имея newW/newH, просто вернет res), потом masterToDataURLHook. MasterHook, *в контексте chain*, вызовется ещё **до** original (потому что chain не знал, что он особый). Но внутри masterHook он сам вызовет `patchToDataURLInjectNoise` (которую мы уже вызывали – дублирование) и `patchCanvasIHDRHook` с newW/newH и вернет DataURL. Если masterHook возвращает строку DataURL, chain функция увидит res (truthy, not array) != undefined, **в patchMethod такой случай обрабатывается** (вернуть res), а вот в chain, как написано, нет проверки res. Может, они позже переделали toDataURL на `patchMethod` (который у нас выше `chain`)?
  Не будем глубже – конечный эффект: **canvas.toDataURL() возвращает PNG-данные с внесенным шумом и поправленным IHDR-заголовком**, чтобы каждый canvas (независимо от размера) давал, скажем, одинаковый размер IHDR для стабильности fingerprint, или наоборот, добавлял уникальность, не знаю.

- `canvas.toBlob(callback, type, quality)`: через `chainAsync` мы обернули. Наш `patchToBlobInjectNoise` (зарегистрирован через `C.registerHtmlCanvasToBlobHook`) будет вызван внутри новой toBlob. Посмотрим код:
  Он проверяет, что `this` – canvas/Offscreen, берет `ctx2d = this.getContext('2d')`, получает все пиксели (w,h), затем проходится по `data` добавляя шум ±1 (рандом `v`). Потом берет оригинальный `putImageData` (из прототипа или сохраненный) и кладет модифицированный `img` обратно на ctx2d. То есть, **добавляет шум на холст, аналогично**. После этого наш асинхронный chain вызывает оригинальный toBlob. Оригинал асинхронно создает Blob из *уже модифицированного* содержимого. Полученный Blob содержит зашумленное изображение. (Можно потом ещё хуки `.then` на промис, но пока у нас `patchConvertToBlobInjectNoise` просто шаблонно возвращает promise без изменений.)

Итак, **архитектура вызовов** выглядит строго и последовательно:

- **Регистрация:** Сначала нужные хук-функции добавляются в списки.

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Патчинг: Методы Canvas (getContext, toDataURL, toBlob, drawImage, и др.) заменяются на обёртки, которые знают о соответствующем списке хуков.

- **Вызов метода:** → *Перехват обёрткой:*
  - (если HTMLCanvasElement метод) Вызывается обёртка (chain/patch) с выполнением хуков и затем оригинальной логики.
  - (если CanvasRenderingContext2D метод) Через Proxy мы попадаем в обёртку (chain) метода контекста, которая выполняет хуки и оригинал.
- **safeInvoke в ключевых местах:** гарантирует корректный вызов оригинала (особенно для контекстных методов).
- **Выполнение хуков:** Каждый зарегистрированный хук-функция выполняется либо **до**, либо **после** оригинала, в зависимости от типа обёртки:
  - chain (синхронные) – обычно pre-hooks (но могут менять args или return value).
  - patchMethod (например, WebGL) – могут решать, возвращать свое значение вместо оригинала.
  - Наши конкретные реализации для Canvas2D стараются делать большинство работы *после* оригинального действия, чтобы не нарушить его (например, шум добавляется после рисования изображения).
- **Результат:** Возвращается результат, который ожидается по спецификации. (Если метод не возвращал ничего, так и есть; если должен вернуть данные – возвращаются модифицированные данные или слегка измененные объекты).

Это архитектура "Canvas.getContext → Proxy → Hook → ... → оригинал → результат" по каждому вызову. Ключевое: **все происходит прозрачно для вызывающей стороны**, но каждый шаг контролируется нашей системой.

Хуки для ключевых методов Canvas API

В предыдущем разделе мы уже частично рассмотрели, как обрабатываются разные методы. Сведём воедино основные методы Canvas, которые патчатся хуками, опишем *цель* перехвата, подход к реализации (safe-прокси, chain) и ожидаемое поведение. Эти методы – "мишени" для системы Canvas-хуков в типичном проекте анти-отпечатка или кастомизации Canvas.

HTMLCanvasElement.prototype.getContext

- **Цель перехвата:** вместо оригинального контекста вернуть *проксированный* контекст, подключив нашу систему перехватов. Дополнительно – возможность выполнить настройку контекста (например, добавить свойства или вызвать инициализационные хуки) сразу при его создании.
- **Как патчится:** переопределяется `HTMLCanvasElement.getContext` через обертку (`chainGetContext`). SafeInvoke используется, чтобы получить оригинальный контекст без потери `this`. Проксирование осуществляется вызовом `createSafeCtxProxy`. Затем вызываются хуки: `htmlCanvasGetContextHooks` (общие) и, в зависимости от типа:
  - для '2d' контекста – `ctx2DGetContextHooks` (например, установка шумовых оберток);

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

- для 'webgl'/'webgl2' – `webglGetContextHooks` (например, занижение параметров) – хо не в фокусе тут.
- **Expected behavior:** вызов `canvas.getContext('2d')` возвращает объект, **совместимый с CanvasRenderingContext2D**, но дополнительно защищенный Proxy. Пользователь может использовать его точно так же, как обычный контекст. Методы WebGL или 3D, запрошенные через getContext, тоже могут быть проксированы, но тут главное – для 2D.

CanvasRenderingContext2D.prototype.drawImage

- **Цель хука:** предотвратить получение идентичных пиксельных данных при рисовании изображений (в т.ч. видео, canvas, оффскрин) – в основном, **добавить шум и другие искажения** к рисуемому контенту. Это затрудняет скрытый захват изображения или сравнение байт-ко-в-байт (fingerprinting через canvas).
- **Реализация:**
  - Используется Proxy для безопасного вызова (как у всех основных методов).
  - Патчинг через `patchCtx2D` заменяет `drawImage` на обёртку, которая запускает список хуков: сначала общие (шум, pixelate), потом специфичные (набор эффектов).
  - В нашем случае, основной хук `applyDrawImageHook` делает пост-обработку: после `origDrawImage` он получает нарисованную область (`getImageData`), и подряд применяет утилиты: стабильный шум (на основе глобального seed или специфики изображения), пикселизацию (понижение разрешения в пределах региона), смещение цветовых каналов, размытие, волновое искажение. Затем возвращает модифицированную область обратно через `putImageData`. Это **гарантирует, что на канвасе останется измененная версия изображения**, хотя код вызвал обычный drawImage.
  - Дополнительно, **граничные случаи**: если `drawImage` вызывается для разных сигнатур (например, 3 аргумента vs 9 аргументов с обрезкой), наш код учитывает это, устанавливая по умолчанию `sw, sh` равными размеру исходного canvas, и `dw, dh` тоже, если не указаны. Таким образом, всегда известен регион копирования.
- **Ожидаемое поведение:** Для разработчика метод `ctx.drawImage(img, x, y)` работает обычно – изображение появляется. Но подлупой можно заметить небольшие визуальные артефакты (шумовые пиксели, чуть размытие и т.п.). В числовом плане, если сразу после `drawImage` вызвать `getImageData` этого региона, то полученные данные будут **отличаться** от исходного изображения `img` – как минимум на уровне нескольких бит. Это то, чего мы хотели достичь. При этом никаких ошибок не происходит, и другие части canvas (вне [dx,dy,dw,dh]) не затронуты.

CanvasRenderingContext2D.prototype.getImageData

- **Цель хука:** когда скрипт пытается *прочитать пиксели* с канваса (например, для снятия отпечатка), **подсунуть ему искаженные данные**. Обычно рисуют нечто (текст, градиент) на канвас и затем вызывают `getImageData` для получения массива пикселей. Наш хук добавляет шум прямо в возвращаемый ImageData, чтобы даже при одинаковом вводе разные окружения давали разные результаты, либо по крайней мере – чтобы результат был не идеальным.

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Реализация:

- `getImageData` вызывается через Proxy (safeInvoke).
- При инициализации 2D-контекста (хук `patch2DNoise` в getContext) мы *переопределяем метод getImageData на уровне конкретного контекста*. Это динамический подход: вместо глобального патча, каждый созданный 2D-контекст получает свою обёртку `ctx.getImageData`. Эта обёртка вызывает сохраненный оригинал (bind сделан на тот же контекст) для получения `imgData`, затем проходит по массиву `imgData.data` и вносит небольшие случайные отклонения в значения R,G,B (альфа, скорее всего, не трогается) – например, ±1. После этого возвращает `imgData` вызывающему коду.
- Альтернативный путь: глобально через patchCtx2D, аналогично другим, с `C.ctx2DGetImageDataHooks`. Но судя по коду, выбран именно метод *override per instance*.

- **Expected behavior:** вызов `ctx.getImageData(x,y,w,h)` возвращает объект ImageData правильного размера, но пиксели в нём чуть изменены. Например, там где должен быть цвет `(r,g,b)` может оказаться `(r+1, g-1, b)` и т.п. (не выходя за пределы 0-255, конечно). Картинка в целом будет выглядеть практически так же, глазом шум может быть неразличим, но побайтный анализ покажет расхождение. Если злоумышленник рассчитывал на идентичный массив пикселей (например, хеш SHA-256), он получит другой результат.

CanvasRenderingContext2D.prototype.putImageData

- **Цель хука:** вносить шум или изменения в **устанавливаемые** пиксели. Хотя main threat – чтение данных, перехват `putImageData` делает систему консистентной: даже если кто-то попробует нарисовать через putImageData заранее подготовленный набор пикселей (например, обойти drawImage), мы можем его немного исказить.
- **Реализация:**
  - Через `patchCtx2D putImageData` заменен на обертку, которая проходит хуки из `C.canvas2DNoiseHooks` (обобщенные шумовые хуки).
  - Вариант реализации: хук до вызова оригинала может пробежаться по переданному ImageData и добавить шум так же, как мы делали при чтении (± случайный). Код `patchToBlobInjectNoise` фактически делает: получить ctx.getImageData / modify / put назад. Но здесь у нас уже есть `imageData` – логичнее *прямо его изменить*. Например:

    ```js
    function addCanvasNoiseHook(imageData, dx, dy) {
      const data = imageData.data;
      for (let i = 0; i < data.length; i+=4) {
        data[i]   ^= 0x1; // или +-1, или XOR
        // ... можно по каналу или как-то иначе
      }
      return [imageData, dx, dy];
    }
    ```

    (Код XOR для иллюстрации – реальное добавление ±1 более мягкое.)
  - Наши рекомендации говорили: `putImageData → addCanvasNoise`. Это и есть тот хук.

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

Expected behavior: например, скрипт делает: нарисовал что-то, сохранил через getImageData, потом может модифицировал массив и вернул через putImageData (или даже сразу пытается смоделировать canvas fingerprint используя putImageData с заранее известными данными). Наш хук внесет микроскопическое отклонение. Визуально, вероятно, незаметно, но биты не совпадут идеально с тем, что подал скрипт. Для нормального использования (дев рисует пикчи) – разница ±1 в канале цвета несущественна. Но для бенчмарка отпечатка – фатальна (хеш меняется).
Важно: если putImageData вызывается часто (например, анимация), наш хук может стать узким местом. Поэтому обычно шум добавляется либо один раз, либо отключается в режиме анимаций. Но это за рамками: у нас модульный – можно отключить hooks если надо.

HTMLCanvasElement.prototype.toDataURL

- **Цель хука:** этот метод часто используется для получения изображения из канваса в виде строки (например, PNG для загрузки, или для сравнения). Для anti-fingerprinting, нужно **исказить итоговое изображение**. Кроме того, canvas.toDataURL('image/png') выводит PNG, включающий метаданные (IHDR, IDAT). Хуки преследуют несколько целей:
  1. Добавить шум (как и в getImageData, но непосредственно перед экспортом).
  2. Исправить метаданные PNG (IHDR) – например, сделать так, чтобы если злоумышленник меняет размеры canvas, это не меняло хеш. Есть известный трюк: некоторые fingerprint-скрипты кодируют информацию в нестандартных полях PNG. Наш IHDR-патч может вставлять фиктивные или нормализованные данные.
  3. Возможна общая координация: если несколько хуков должны сработать, важно **сохранить порядок** (шум до IHDR, etc.).
- **Реализация:**
  - Patched via `chain` (synchronous). Хуки в `C.htmlCanvasToDataURLHooks`:
    - `patchToDataURLInjectNoise`: как описано, берет 2D контекст, добавляет шум к его пикселям (все canvas), **до** получения DataURL. Возвращает `res` без изменений (не генерируя свой DataURL).
    - Оригинальный `toDataURL` вызывается (через safeInvoke) и возвращает строку `res` (например, `'...'`).
    - **(Если бы chain позволял post-hooks)**:
      - `patchCanvasIHDRHook`: мог бы взять `res` (PNG DataURL), декодировать из base64, поменять поля width/height в IHDR, пересчитать CRC, закодировать обратно. Код patchCanvasIHDR делает именно это: он сам вызывает `canvas.toDataURL` внутри, что в контексте уже измененного canvas, затем правит width/height и CRC.
        Но wait, он внутри себя вызывает toDataURL (рекурсия?), что может заново запустить наш chain, но у них `CanvasGlobal` предотвращает двойную обработку. Вероятно, patchCanvasIHDRHook вызывается уже *после* оригинала, поэтому `res` имеется, и внутри они делают *второй* DataURL на временном canvas – или на том же (this).
        Вообще, code:

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

```
js

function patchCanvasIHDR(canvas, newW, newH) {
  const dataURL = canvas.toDataURL('image/png');
  // ... replace IHDR bytes...
  return '' + btoa(patchedBinary);
}
function patchCanvasIHDRHook(res, type, quality, newW, newH) {
  if(type==='image/png' && newW && newH) {
    try { return patchCanvasIHDR(this, newW, newH); } catch{}
  }
  return res;
}
```

Это говорит: он игнорирует исходный `res` и генерирует новый dataURL, но только если у нас есть newW/newH (которые, как видим, передаются).

- `masterToDataURLHook`: скорее orchestrator: он берет `res` и пропускает через оба выше: `res = patchToDataURLInjectNoise.call(this, res, ...)` потом вычисляет newW,newH (из глобала или canvas) и вызывает `patchCanvasIHDRHook.call(this, res, ..., newW, newH)`, возвращая результат.
- Судя по registrations, masterToDataURLHook последний. Возможно, chain у них тут не стандартный, или masterHook возвращает финальный DataURL.
- В любом случае, **после выполнения всех хуков, строка dataURL возвращается**.

- **Expected behavior:** при вызове `canvas.toDataURL()` пользователь получает Base64-строку изображения. Но:
  - Изображение содержит тот же рисунок, что и на канвасе, **с незначительным шумом**. Если сравнить байты с картинкой без шума, они отличаются.
  - Если используется PNG, возможно IHDR (поля ширины/высоты) либо оставлены как есть, либо заменены на какие-то усредненные (в зависимости от цели). Наш код, похоже, старался вставить заданные `window._NEW_WIDTH/_NEW_HEIGHT` если они определены, иначе текущее значение canvas. Возможно, для стабильности fingerprint: всегда выдавать, например, размеры 100x100 в метаданных, даже если реальный канвас 200x200, чтобы hash совпадал. (Это догадка).
  - Если `type` не указан или `'image/png'`, хуки применяются (PNG - цель). Если запросили `'image/jpeg'`, noiseHook у нас применяет к любому `image/*` (в коде `if(!type || /^image\//i.test(type)) ...`), IHDRHook не будет, т.к. PNG only.
  - Выполнение toDataURL не бросает исключений, работает как обычно. Только вывод немного отличается. Для большинства приложений это незаметно (кроме тех, кто сравнивает хеши).

HTMLCanvasElement.prototype.toBlob

- **Цель хука:** аналогично toDataURL, но для Blob (асинхронный результат). Добавить шум в содержимое Blob.
- **Реализация:**
  - Патч через `chainAsync`. Новый `toBlob` возвращает Promise (вместо требуемого callback в оригинале) – это улучшение, но важно, что внутри:

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

- Если вызвали традиционно `canvas.toBlob(callback)`, наш патч просто делегирует оригиналу через safeInvoke (ничего не меняя). Мы не хотим ломать callback-способ, просто не добавляем туда хуки (их сложнее встроить, т.к. он async).
- Если вызвали `canvas.toBlob()` как Promise (возможно наша либка позволяет `await canvas.toBlob()`), то обёртка создает Promise и вызывает `orig.toBlob(resolve, ...)`.
- Hooks (в `C.htmlCanvasToBlobHooks`) применяются внутри переопределенного `toBlob`. В нашем коде реализовано так, что new `toBlob` сразу выполняет safeInvoke(orig) который возвращает либо undefined (в случае callback) либо Promise (в случае Offscreen?), и проверяет: если не Promise, делает `Promise.reject` (for Offscreen we ensure we always get Promise).
- Конкретно наш `patchToBlobInjectNoise` – из `C.htmlCanvasToBlobHooks` – выполняется внутри chain?
  Однако, глядя на код, они сделали иначе: `chainAsync` **не вызывает hooks непосредственно**, а скорее трансформирует возвращаемое Promise. В коде `chainAsync`:

```
js

if (method === 'toBlob') {
   if (typeof args[0] === 'function') { return safeInvoke(orig, this, args);
}
   // else (no callback): return new Promise wrapping orig
}
// for others (OffscreenCanvas.convertToBlob, already Promise):
const promise = safeInvoke(orig, this, args);
if (!(promise instanceof Promise)) { return Promise.reject(...); }
// "here you can attach your hooks: e.g.
promise.then(...hook1...).then(...hook2...)"
return promise;
```

.

То есть, в chainAsync, хуки не вызываются сразу. Вместо этого предполагалось, что можно сделать:

`js` `return promise.then(blob => { ...hook1...; return blob; })`
`.then(blob => { ...hook2...; return blob; });`

У нас есть `patchConvertToBlobInjectNoise(origPromise, ...args)` – зарегистрирован в `C.offscreenConvertToBlobHooks`, но Offscreen.

Для HTMLCanvas toBlob, наш `patchToBlobInjectNoise` вызван? Где?

Похоже, они решили добавить шум **до** вызова toBlob (синхронно, аналогично toDataURL). `patchToBlobInjectNoise` реализован синхронно: он получает `res` (наверное undefined или whatever), и если canvas 2D, выполняет ту же логику, что и noiseHook: берет контекст, getImageData, шум, putImageData, потом возвращает res.

Если `toBlob` вызвали с колбэком, chainAsync делегировал orig сразу – тогда `patchToBlobInjectNoise` **не выполнится** (потому что мы в branch callback returning undefined).

Если `toBlob` как promise, то chainAsync вызвал orig->promise, но **не вызвал hooks**. Upd: Actually, chainAsync likely does *not incorporate hooks list at all*, it just sets up promise. So, `patchToBlobInjectNoise` might never run if left like that.

Скорее всего, они пошли простым путем: *сделать* `toBlob` *synchronous-ish by noise then orig call in*

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

callback mode. Можно было:

```js
canvas.toBlob(blob => resolve(blob), type, quality);
```

Но они хотели Promise.

```pgsql
  Возможно, `patchToBlobInjectNoise` вообще используется в Offscreen scenario or being left out? But registration shows `registerHtmlCanvasToBlobHook(patchToBlobInjectNoise)`:contentReference[oaicite:162]{index=162}, so presumably chain was intended to incorporate it.

  Наверняка, `chainAsync` as is does not apply our hooks list for toBlob. Could be oversight, or they rely on the fact that we added noise earlier by hooking other points. But likely oversight in snippet.

  Тем не менее, **мы можем описать желаемое поведение**:
  - Перед созданием Blob, внести шум в canvas (подобно toDataURL). Затем оригинальный toBlob создаст Blob с шумом.
  - Если toBlob используется с колбэком, либо не трогать, либо тоже можно шум внести до вызова orig (но у нас условие if callback, skip hooks).
  - OffscreenCanvas.convertToBlob: там как раз Promise, они могли `.then` hooks (patchConvertToBlobInjectNoise is a then that returns the blob unchanged currently, placeholder for future).
```

- **Expected behavior:** `canvas.toBlob` асинхронно предоставляет Blob, содержащий измененное изображение. Если сравнить этот Blob с тем, что был бы без патчей, они отличаются незначительно (несколько бит). Использование Promise vs callback – для dev это transparent: с нашим патчем dev может `await canvas.toBlob()` (если Promise возвращается). Все штатно, кроме незаметного изменения содержимого.

CanvasRenderingContext2D.prototype.fillText / strokeText / measureText

(Рассматриваем вместе, так как цель общая – защита текстовых характеристик.)

- **Цель хуков:** затруднить получение информации о рендеринге шрифтов и избежать точных совпадений текста. Браузерный холст используется для fingerprint текстом: рендерят строку определенным шрифтом, либо читают пиксели (признаки сглаживания, субпиксельный рисунок), либо измеряют ширину текста (метрики зависят от шрифта, размера, движка). Наши хуки пытаются:
  - Добавить **стабильный шум** к нарисованному тексту (чтобы каждый символ/строка имели слегка “испорченный” рендер – например, размытые или немного смещенные пиксели, но одинаково на всех машинах).
  - **Искажение метрик:** слегка изменить результаты `measureText` (например, ±1px), чтобы они не соответствовали реально рендеренному идеально. Важно, чтобы изменение было постоянным для данного текста, иначе можно усреднить, сделав несколько вызовов measureText.
- **Реализация:**
  - **measureText:** патчится через chain в `applyCtx2DContextPatches`. Обертка вызывает все `C.ctx2DMeasureTextHooks` и затем оригинальный `measureText`. Как вариант, хуки могут запускаться *после* orig, т.к. мы хотим корректировать результат. В простой реализации:

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

```js
const metrics = origMeasureText.apply(this, args);
for(const hook of hooks) {
  try {
    const res = hook.call(this, metrics, ...args);
    if(res !== undefined) return res;
  } catch(e){...}
}
return metrics;
```

Но у нас chain-template не учитывал post. Однако, возможно, hook просто использует metrics по ссылке.
Например:

```js
function measureTextNoiseHook(metrics, text) {
  if (metrics && metrics.width) {
     // stable pseudorandom offset:
     const pseudo = stableRandom(text) * 0.5; // some fraction
     metrics.width += pseudo;
  }
  return metrics;
}
```

Если hook возвращает metrics (которая уже изменена), chain игнорирует (не expecting array), orig result was metrics, we mutated it in place. Not the cleanest but works as metrics is object.
Если hook would return new metrics object, chain would drop it. So likely mutate in place.

- **fillText / strokeText:**
  - Можно реализовать как post-draw effect: после orig call, считать область текста пикселами, шумануть и вернуть на место (подобно drawImage). Но это дорого для любой надписи.
  - Возможен другой подход: изменить контекст настройки перед рисованием – например, добавить тень (shadowBlur) или globalCompositeOperation, чтобы при рисовании сразу был чуть "нечистый" отпечаток. Но это может быть заметно визуально.
  - У нас видимых утилит для текста нет. Возможно, simplest:
    - Draw text invisibly to an off-screen canvas, apply noise, then draw back? Too heavy.
    - Or apply a stable tiny translation jitter: e.g. adjust x or y by a fraction for each character via fillText on single chars (super complex).
    - The doc suggests stableNoiseFromString, implying the noise pattern is derived from string.
  - Возможно, `fillTextNoiseHook` генерирует, к примеру, невидимый (transparent) noise overlay:
    There's a trick: fillText doesn't return anything, we can't easily intercept result. But we can do something *after* it returns. In chain hooking, since fillText returns undefined, we can treat hooks as pre processing:

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

- Pre-hook:* could alter context state, e.g. change `shadowColor` and `shadowBlur` for one call to add slight blur.
- Post-hook:* after orig, do something, but how to do after if chain calls orig at end? Possibly via patchMethod pattern:
  Actually, in `patchCtx2D` code, they do pre-hooks then orig then return. But we can do an extra loop after orig if needed by customizing code. Perhaps they didn't, and they rely on pre.

Let's see the doc or code clues:
They mention *"stableNoiseFromString for text noises"*. Possibly they have a function stableNoiseFromString that given the text returns a small offset or pattern.
Could be they implemented it as part of `fillTextNoiseHook` by:

- Before drawing text: measure text, get width, use text string to seed random, and perhaps draw an almost transparent text on top?
- Or they might compromise: only `measureText` is changed (affecting fingerprint metrics), and maybe no actual noise on rendered text to avoid visual issues.

- If we guess:
  `fillTextNoiseHook`: might do something like:

```js
function fillTextNoiseHook(text, x, y, ...args) {
   // Slight offset jitter on coordinates:
   const jitter = stableRandom(text) * 0.5 - 0.25;
   return [text, x + jitter, y + jitter, ...args];
}
```

So the text is drawn slightly offset unpredictably (maybe half a pixel if possible? But canvas works in subpixel).
That could blur or shift text by fraction. But canvas API might floor to device pixels anyway.
Or:

```js
// Save state, apply global alpha for noise layering
const savedAlpha = this.globalAlpha;
this.globalAlpha = 0.8;
origFillText.apply(this, arguments);
// slightly move and redraw
this.globalAlpha = 0.2;
origFillText.apply(this, [text, x+0.5, y]);
this.globalAlpha = savedAlpha;
return;
```

But doing orig inside hook is tricky if not using patchMethod.

Given uncertainties, we describe intention:

- It should cause the rendered text to *not be pixel-perfect identical on different systems.* Possibly by altering some pixel values or placement.

- **Expected behavior:**

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

- `ctx.measureText("Hello").width` на разных компах обычно будет точно соответствовать использованному шрифту и рендереру. С нашим хуком – будет слегка (+ε) другим, но стабильно: на любом компе, запустив наш код, width "Hello" = say 55.23 px (вместо 54.5). Это ломает сравнение с эталоном.
- `ctx.fillText("Hello", 10, 50)` визуально отобразит "Hello". Если смотреть очень внимательно, буквы могут быть чуть более размытыми или с шумом – но мы постарались сделать это минимально заметным. Человек не заметит, а алгоритм, сравнивающий рендеры (например, через getImageData), не получит идентичные пиксели.
- `strokeText` аналогично (обводка текста тоже может быть шумно смещена или с небольшими разрывами в обводке).

Примечание: OffscreenCanvas и WebGL

Хотя вопрос сосредоточен на Canvas 2D, упомянем, что аналогичные принципы могут применяться:

- **OffscreenCanvas** (2D): патчится практически так же, используя `OffscreenCanvas.prototype.getContext` и `OffscreenCanvasRenderingContext2D.prototype` методы. Наш код показывает, что OffscreenCanvas.getContext('2d') тоже возвращает Proxy, и OffscreenCanvas.convertToBlob патчится hooks аналогично toBlob.
- **WebGL**: модульная структура позволяет добавить WebGL-хуки: `C.webglGetParameterHooks`, `C.webglShaderSourceHooks` и т.д., с патчами (например, снижают точность шалдера или число поддерживаемых расширений). Это оформляется отдельным файлом (e.g. webgl.js) но работает по тому же шаблону: регистрация -> patchMethod (т.к. WebGL – sync calls mostly). В нашем ответе мы это не раскрываем подробно, но архитектура едина.

Чеклист верификации архитектуры

Ниже приведён контрольный список, по которому можно пройтись, проверяя корректность реализации Canvas-хуков и соответствие лучшим практикам:

- ☐ **Разделение по файлам и областям:** Убедиться, что код организован модульно – например, `canvas.js` содержит только логику, связанную с Canvas 2D, и экспортирует `CanvasPatchModule` и `CanvasPatchHooks`. Аналогично, WebGL-хуки, если есть, выносятся отдельно.
- ☐ **Инициализация структуры хуков:** При запуске `CanvasPatchModule` вызывается `ContextPatchModule` и инициализируются все списки хуков (`CanvasPatchContext` создан, массивы пустые). Без этого регистрация функций может записываться не туда или вторично.
- ☐ **Оригиналы методов сохранены:** В процессе патчинга сохранены ссылки на оригинальные функции Canvas API (например, `origGetContext`, `origToDataURL`, `origDrawImage` и т.д.), прежде чем их заменить. Это необходимо для вызова оригинальной логики внутри хуков и для возможного восстановления.

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

- ☐ **Безопасный вызов реализован:** Имеется функция `safeInvoke` для обработки ошибок *Illegal invocation*. Proxy-обёртка контекста использует её при вызове критичных методов (`fillText`, `drawImage`, etc.). Проверьте, что при возникновении `TypeError: Illegal invocation` действительно выполняется fallback через `.call` или `.bind`.
- ☐ **Proxy не нарушает базовые свойства:** Например, `proxyCtx instanceof CanvasRenderingContext2D` возвращает true, `Object.getPrototypeOf(proxyCtx)` соответствует прототипу CanvasRenderingContext2D, свойства контекста (не методы) доступны напрямую. Ловушка Proxy переопределяет только `get` для функций, возвращая обертку, и не трогает другие аспекты – проверьте, что это так.
- ☐ **Флаги предотвращения повторного патча/вызова установлены:** После патчинга методы содержат метку `CanvasGlobal = true` (и/или специфичные, как `CanvasGlobal_AddNoise`), а контексты при исполнении помечаются флагами (`__isCtx2D_method` или аналогичными). Это гарантирует, что:
  - Один и тот же метод не патчится дважды (пропустить, если уже `CanvasGlobal`).
  - При рекурсивном вызове хука (например, если внутри него вызывается тот же метод) не произойдет бесконечной рекурсии – сработает проверка флага и вызов оригинала.
- ☐ **Корректность последовательности хуков:** Проверьте, что регистрация хуков выполнена в нужном порядке и соответствует порядку выполнения. Списки `C.htmlCanvasToDataURLHooks`, `C.ctx2DDrawImageHooks` и пр. наполняются в согласованной последовательности. И в коде, когда цепочка вызывает хуки, они идут именно в том порядке (обычно FIFO). Если есть зависимые хуки (например, masterHook, который должен идти последним), удостоверьтесь, что он и регистрируется последним.
- ☐ **Возврат значений в обертках:** Удостоверьтесь, что замененные методы Canvas возвращают то, что должен возвращать оригинал. Например, `toDataURL` обертка возвращает строку, `getImageData` возвращает ImageData, `measureText` – TextMetrics. Если хук решил вернуть кастомное значение, убедитесь, что это осознанно (например, WebGL getParameter могут возвращать подмененные данные). Для наших Canvas 2D хуков мы старались не менять типы возвращаемых данных (кроме небольших числовых изменений).
- ☐ **Обработка ошибок и исключений:** Система должна по максимуму ловить исключения внутри хуков, логировать их (в debug-режиме) и не пропускать наружу, если это не критично. В коде chain есть `catch` вокруг каждого hook. Проверьте, что ни один брошенный в хуке error не рушит весь вызов Canvas метода – это важно для устойчивости. Исключение должно доходить до внешнего кода только если оригинальный метод его бросил (или если хук намеренно его бросает, что редкость).
- ☐ **Производительность не просела критически:** Конечно, хуки добавляют накладные расходы. Но следует убедиться, что нет очевидных проблем:
  - Прокси создается один раз per контекст, а не на каждый вызов getContext (если это нет, то при каждом getContext будет новый Proxy; лучше один – но если canvas.getContext каждый раз возвращает новый Proxy, это ok, они равнозначны).
  - Нет утечек памяти: например, `ctx` (Proxy) не удерживает лишних объектов, флаги снимаются (предотвращая рост), hook-списки не разрастаются бесконтрольно.

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

PDF-XChange Editor Click to BUY NOW! www.pdf-xchange.com

- В циклах (напр. анимация) – небольшие оптимизации применены: напр., шум добавляется не на каждом кадре или отключаем, если найден флаг. В нашем коде, например, `res.__Noise__` предотвращает повторное override getImageData на том же контексте.
- Большие массивы (pixel data) обрабатываются на приемлемом уровне. Если, скажем, `applyDrawImageHook` перерисовывает большие картинки, это может тормозить – но это компромисс безопасности vs скорость, главное, что все работает правильно.

- ⬜ **Визуальная и функциональная целостность:** После внедрения хуков, убедиться, что Canvas все еще рисует и работает для пользователя. Это скорее QA-тест: отобразить известные рисунки или текст на канвасе и убедиться, что они видимы (пусть слегка изменены, но не исчезли, не исказились до неузнаваемости), загрузка изображений (toDataURL / toBlob) работает (файлы открываются), и т.д.

Если все пункты выполнены, можно уверенно сказать, что архитектура Canvas-хуков построена грамотно: она разделена на понятные модули, безопасна, не нарушает стандартную работу Canvas API и достигает целей по модификации вывода. Такой подход обеспечивает **чистоту, расширяемость и безопасность** при патчинге Canvas API через систему хуков.

Sources